

CHƯƠNG 8. GIẢM CHIỀU DỮ LIỆU

Nhiều bài toán học máy liên quan đến hàng nghìn hoặc thậm chí hàng triệu đặc trưng cho mỗi trường hợp huấn luyện. Tất cả các đặc trưng này không chỉ làm cho quá trình huấn luyện cực kỳ chậm mà còn có thể làm cho việc tìm ra một giải pháp tốt trở nên khó khăn hơn nhiều, như bạn sẽ thấy. Vấn đề này thường được gọi là **lời nguyền của số chiều**.

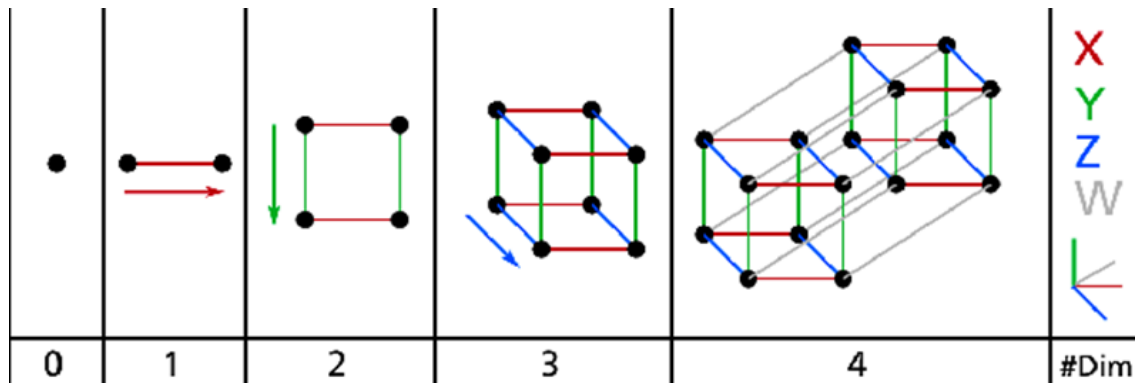
May mắn thay, trong các bài toán thực tế, thường có thể giảm đáng kể số lượng đặc trưng, biến một bài toán không thể giải quyết được thành một bài toán có thể giải quyết được. Ví dụ, hãy xem xét các hình ảnh MNIST (được giới thiệu trong Chương 3): các pixel ở đường biên hình ảnh hầu như luôn luôn trắng, vì vậy bạn có thể loại bỏ hoàn toàn các pixel này khỏi tập huấn luyện mà không mất nhiều thông tin. Như chúng ta đã thấy trong chương trước (Hình 7-6), điều này xác nhận rằng các pixel này hoàn toàn không quan trọng đối với tác vụ phân loại. Ngoài ra, hai pixel lân cận thường có tương quan cao: nếu bạn hợp nhất chúng thành một pixel duy nhất (ví dụ: bằng cách lấy trung bình cường độ của hai pixel), bạn sẽ không mất nhiều thông tin.

Ngoài việc tăng tốc quá trình huấn luyện, giảm chiều dữ liệu cũng cực kỳ hữu ích cho việc trực quan hóa dữ liệu. Giảm số lượng chiều xuống còn hai (hoặc ba) giúp có thể vẽ một cái nhìn cô đọng về một tập huấn luyện có chiều cao trên một biểu đồ và thường thu được một số hiểu biết quan trọng bằng cách phát hiện các mẫu một cách trực quan, chẳng hạn như các cụm. Hơn nữa, trực quan hóa dữ liệu là điều cần thiết để truyền đạt kết luận của bạn cho những người không phải là nhà khoa học dữ liệu—đặc biệt là những người ra quyết định sẽ sử dụng kết quả của bạn.

Trong chương này, chúng ta sẽ thảo luận đầu tiên về lời nguyền của số chiều và có được cảm nhận về những gì xảy ra trong không gian có chiều cao. Sau đó, chúng ta sẽ xem xét hai cách tiếp cận chính để giảm chiều (phép chiếu và học đa tạp), và chúng ta sẽ đi qua ba trong số các kỹ thuật giảm chiều phổ biến nhất: PCA, phép chiếu ngẫu nhiên và nhúng tuyến tính cục bộ (LLE).

Lời nguyền của số chiều

Chúng ta đã quá quen với việc sống trong không gian ba chiều đến nỗi trực giác của chúng ta thất bại khi chúng ta cố gắng tưởng tượng một không gian có chiều cao. Ngay cả một siêu hình lập phương 4D cơ bản cũng cực kỳ khó hình dung trong tâm trí chúng ta (xem Hình 8-1), chứ đừng nói đến một hình elip 200 chiều bị uốn cong trong không gian 1.000 chiều.



Hình 8-1. Điểm, đoạn thẳng, hình vuông, hình lập phương và tesseract (siêu hình lập phương từ 0D đến 4D)

Hóa ra nhiều thứ hoạt động rất khác nhau trong không gian có chiều cao. Ví dụ, nếu bạn chọn một điểm ngẫu nhiên trong một hình vuông đơn vị (một hình vuông 1×1), nó sẽ chỉ có khoảng 0.4% cơ hội nằm cách đường biên dưới 0.001 (nói cách khác, rất khó có khả năng một điểm ngẫu nhiên sẽ “cực đoan” dọc theo bất kỳ chiều nào). Nhưng trong một siêu hình lập phương đơn vị 10.000 chiều, xác suất này lớn hơn 99.999999%. Hầu hết các điểm trong một siêu hình lập phương có chiều cao đều rất gần đường biên.

Đây là một sự khác biệt rắc rối hơn: nếu bạn chọn hai điểm ngẫu nhiên trong một hình vuông đơn vị, khoảng cách giữa hai điểm này sẽ, trung bình, khoảng 0.52. Nếu bạn chọn hai điểm ngẫu nhiên trong một hình lập phương đơn vị 3D, khoảng cách trung bình sẽ là khoảng 0.66. Nhưng còn hai điểm được chọn ngẫu nhiên trong một siêu hình lập phương đơn vị 1.000.000 chiều thì sao? Khoảng cách trung bình, dù bạn tin hay không, sẽ là khoảng 408.25 (khoảng $\sqrt{1.000.000/6}$)! Điều này phản trực giác: làm thế nào hai điểm có thể cách xa nhau đến vậy khi cả hai đều nằm trong cùng một siêu hình lập phương đơn vị? Vâng, có rất nhiều không gian trong các chiều cao. Kết quả là, các tập dữ liệu có chiều cao có nguy cơ rất thưa thớt: hầu hết các trường hợp huấn luyện có thể sẽ cách xa nhau. Điều này cũng có nghĩa là một trường hợp mới có thể sẽ cách xa bất kỳ trường hợp huấn luyện nào, làm cho dự đoán kém tin cậy hơn nhiều so với trong các chiều thấp hơn, vì chúng sẽ dựa trên các phép ngoại suy lớn hơn nhiều. Tóm lại, tập huấn luyện có càng nhiều chiều, nguy cơ quá khớp càng lớn.

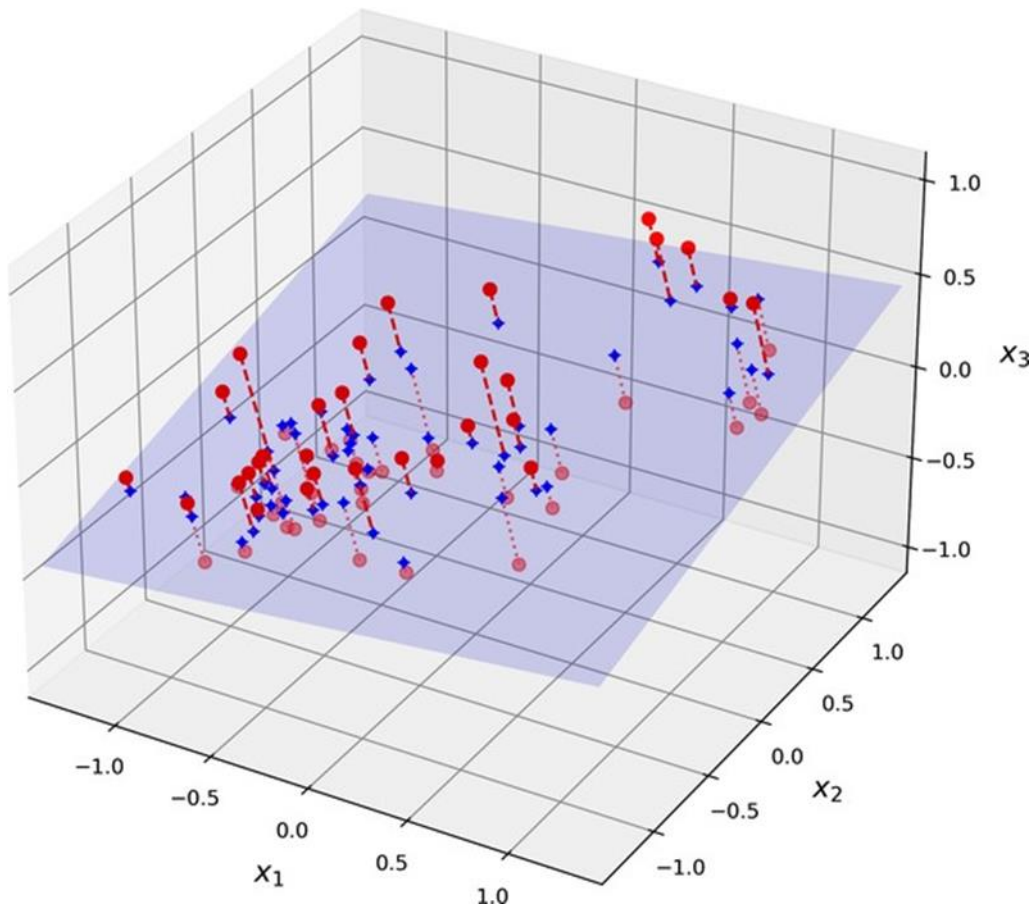
Về lý thuyết, một giải pháp cho lời nguyền của số chiều có thể là tăng kích thước của tập huấn luyện để đạt được mật độ đủ của các trường hợp huấn luyện. Thật không may, trong thực tế, số lượng trường hợp huấn luyện cần thiết để đạt được mật độ nhất định tăng theo cấp số mũ với số lượng chiều. Với chỉ 100 đặc trưng—ít hơn đáng kể so với trong bài toán MNIST—tất cả đều có giá trị từ 0 đến 1, bạn sẽ cần nhiều trường hợp huấn luyện hơn số nguyên tử trong vũ trụ quan sát được để các trường hợp huấn luyện nằm trong khoảng 0.1 của nhau trung bình, giả sử chúng được trải đều trên tất cả các chiều.

Các cách tiếp cận chính để giảm chiều

Trước khi chúng ta đi sâu vào các thuật toán giảm chiều cụ thể, hãy xem xét hai cách tiếp cận chính để giảm chiều: phép chiếu và học đa tạp.

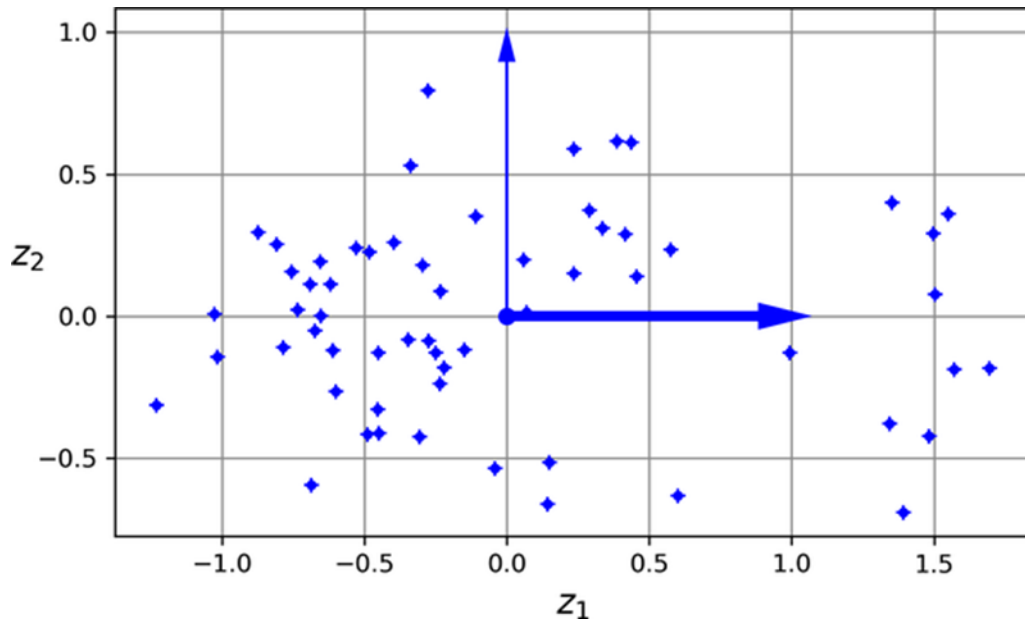
Phép chiếu

Trong hầu hết các bài toán thực tế, các trường hợp huấn luyện không được trải đều trên tất cả các chiều. Nhiều đặc trưng gần như không đổi, trong khi những đặc trưng khác có tương quan cao (như đã thảo luận trước đó cho MNIST). Kết quả là, tất cả các trường hợp huấn luyện nằm trong (hoặc gần) một không gian con có chiều thấp hơn nhiều của không gian có chiều cao. Điều này nghe có vẻ rất trừu tượng, vì vậy hãy xem một ví dụ. Trong Hình 8-2, bạn có thể thấy một tập dữ liệu 3D được biểu diễn bằng các hình cầu nhỏ.



Hình 8-2. Một tập dữ liệu 3D nằm gần một không gian con 2D

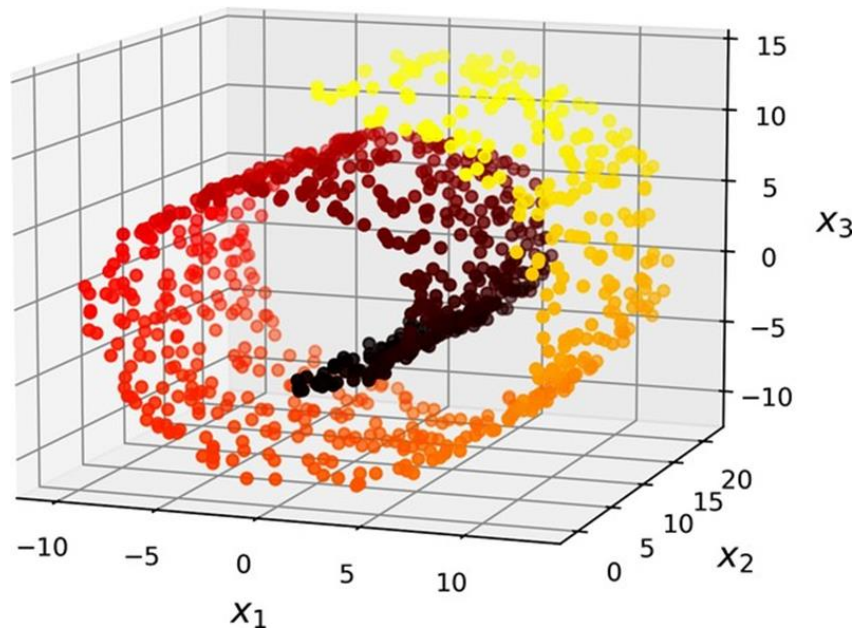
Lưu ý rằng tất cả các trường hợp huấn luyện nằm gần một mặt phẳng: đây là một không gian con có chiều thấp hơn (2D) của không gian có chiều cao hơn (3D). Nếu chúng ta chiếu mọi trường hợp huấn luyện một cách vuông góc lên không gian con này (như được biểu diễn bằng các đường nét đứt ngắn nối các trường hợp với mặt phẳng), chúng ta sẽ nhận được tập dữ liệu 2D mới được hiển thị trong Hình 8-3. Ta-da! Chúng ta vừa giảm chiều của tập dữ liệu từ 3D xuống 2D. Lưu ý rằng các trục tương ứng với các đặc trưng mới z_1 và z_2 : chúng là tọa độ của các phép chiếu trên mặt phẳng.



Hình 8-3. Tập dữ liệu 2D mới sau phép chiếu

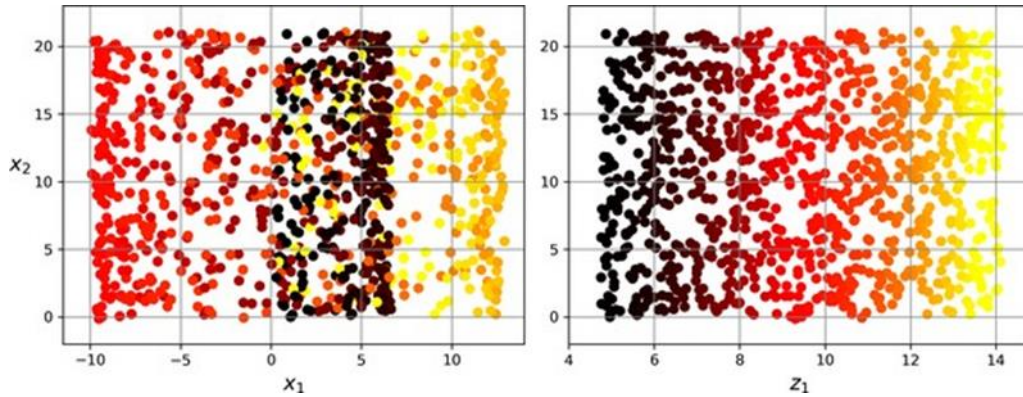
Học đa tạp (Manifold Learning)

Tuy nhiên, phép chiếu không phải lúc nào cũng là cách tiếp cận tốt nhất để giảm chiều. Trong nhiều trường hợp, không gian con có thể xoắn và uốn cong, như trong tập dữ liệu đồ chơi Swiss roll nổi tiếng được biểu thị trong Hình 8-4.



Hình 8-4. Tập dữ liệu Swiss roll

Đơn giản là chiếu lên một mặt phẳng (ví dụ, bằng cách bỏ đi x_3) sẽ làm dẹt các lớp khác nhau của Swiss roll lại với nhau, như trong hình bên trái của Hình 8-5. Điều bạn có lẽ muốn thay vào đó là mở cuộn Swiss roll để có được tập dữ liệu 2D ở phía bên phải của Hình 8-5.

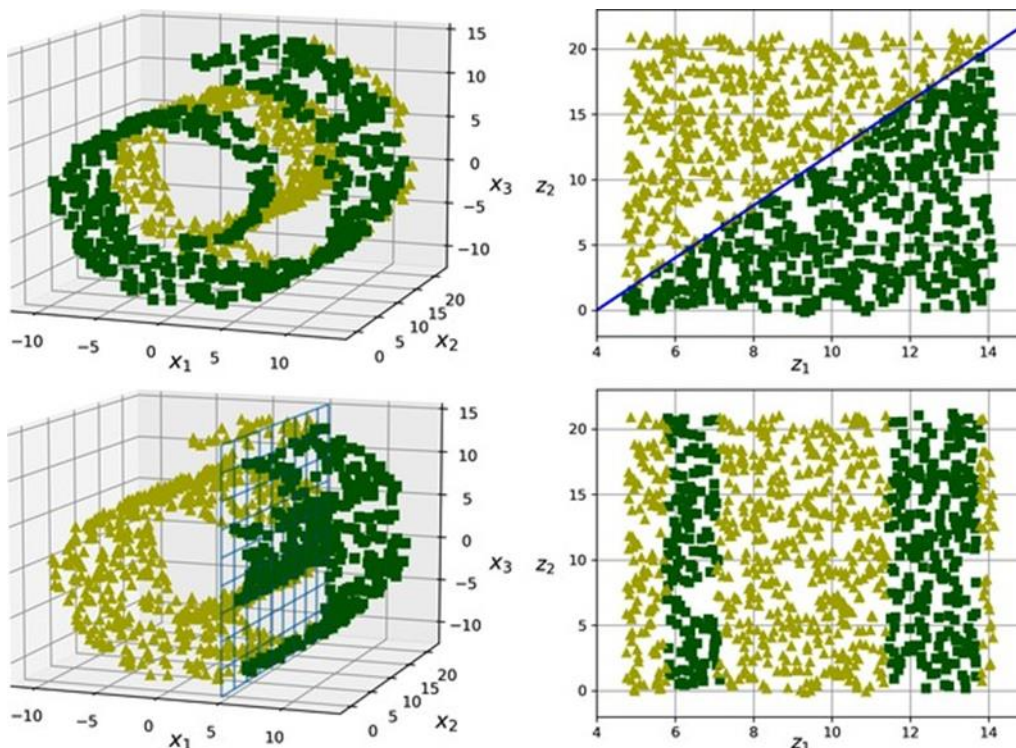


Hình 8-5. Làm dẹt bằng cách chiếu lên một mặt phẳng (trái) so với mở cuộn Swiss roll (phải)

Swiss roll là một ví dụ về một **đa tạp 2D**. Nói một cách đơn giản, một đa tạp 2D là một hình dạng 2D có thể bị uốn cong và xoắn trong một không gian có chiều cao hơn. Tổng quát hơn, một **đa tạp d -chiều** là một phần của không gian n -chiều (trong đó $d < n$) mà cục bộ giống một siêu phẳng d -chiều. Trong trường hợp của Swiss roll, $d = 2$ và $n = 3$: nó cục bộ giống một mặt phẳng 2D, nhưng nó được cuộn trong chiều thứ ba. Nhiều thuật toán giảm chiều hoạt động bằng cách mô hình hóa đa tạp mà các trường hợp huấn luyện nằm trên đó; đây được gọi là **học đa tạp**. Nó dựa trên **giả định đa tạp**, còn được gọi là **giả thuyết đa tạp**, cho rằng hầu hết các tập dữ liệu thực tế có chiều cao đều nằm gần một đa tạp có chiều thấp hơn. Giả định này rất thường được quan sát thực nghiệm. Một lần nữa, hãy nghĩ về tập dữ liệu MNIST: tất cả các hình ảnh chữ số viết tay đều có một số điểm tương đồng. Chúng được tạo thành từ các đường nối liền, các đường biên màu trắng và chúng ít nhiều được căn giữa. Nếu bạn tạo hình ảnh ngẫu nhiên, chỉ một phần rất nhỏ trong số chúng sẽ trông giống các chữ số viết tay. Nói cách khác, số bậc tự do có sẵn cho bạn nếu bạn cố gắng tạo một hình ảnh chữ số thấp hơn đáng kể so với số bậc tự do bạn có nếu bạn được phép tạo bất kỳ hình ảnh nào bạn muốn. Những ràng buộc này có xu hướng nén tập dữ liệu vào một đa tạp có chiều thấp hơn. Giả định đa tạp thường đi kèm với một giả định ngầm khác: rằng tác vụ đang thực hiện (ví dụ: phân loại hoặc hồi quy) sẽ đơn giản hơn nếu được biểu diễn trong không gian chiều thấp hơn của đa tạp. Ví dụ, trong hàng trên cùng của Hình 8-6, Swiss roll được chia thành hai lớp: trong không gian 3D (bên trái) đường biên quyết định sẽ khá phức tạp, nhưng trong không gian đa tạp 2D đã mở cuộn (bên phải) đường biên quyết định là một đường thẳng.

Tuy nhiên, giả định ngầm này không phải lúc nào cũng đúng. Ví dụ, trong hàng dưới cùng của Hình 8-6, đường biên quyết định nằm ở $x_1 = 5$. Đường biên quyết định này trông rất đơn giản trong không gian 3D gốc (một mặt phẳng dọc), nhưng nó trông phức tạp hơn trong đa tạp đã mở cuộn (một tập hợp bốn đoạn thẳng độc lập). Tóm lại, giảm chiều của tập huấn luyện trước khi huấn luyện một mô hình thường sẽ tăng tốc quá trình huấn luyện, nhưng nó không phải

lúc nào cũng dẫn đến một giải pháp tốt hơn hoặc đơn giản hơn; tất cả phụ thuộc vào tập dữ liệu. Hy vọng bạn bây giờ đã hiểu rõ về lời nguyên của số chiều là gì và các thuật toán giảm chiều có thể chống lại nó như thế nào, đặc biệt khi giả định đa tạp đúng. Phần còn lại của chương này sẽ đi qua một số thuật toán phổ biến nhất để giảm chiều.



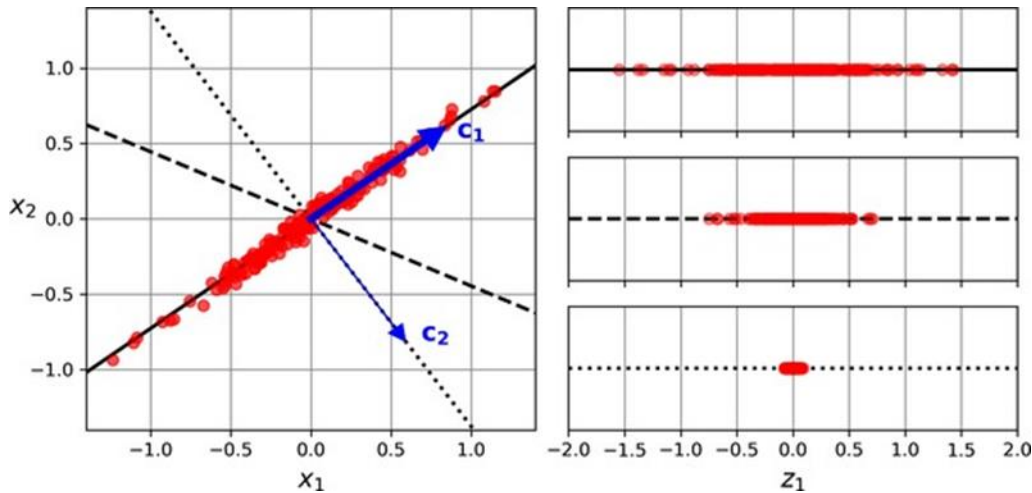
Hình 8-6. Đường biên quyết định không phải lúc nào cũng đơn giản hơn với các chiều thấp hơn

PCA

Phân tích thành phần chính (PCA) cho đến nay là thuật toán giảm chiều phổ biến nhất. Đầu tiên, nó xác định siêu phẳng nằm gần dữ liệu nhất, và sau đó nó chiếu dữ liệu lên đó, giống như trong Hình 8-2.

Bảo toàn phương sai

Trước khi bạn có thể chiếu tập huấn luyện lên một siêu phẳng có chiều thấp hơn, trước tiên bạn cần chọn siêu phẳng phù hợp. Ví dụ, một tập dữ liệu 2D đơn giản được biểu diễn ở bên trái trong Hình 8-7, cùng với ba trục khác nhau (tức là siêu phẳng 1D). Ở bên phải là kết quả của phép chiếu tập dữ liệu lên mỗi trục này. Như bạn có thể thấy, phép chiếu lên đường liền nét bảo toàn phương sai tối đa (trên cùng), trong khi phép chiếu lên đường chấm chấm bảo toàn rất ít phương sai (dưới cùng) và phép chiếu lên đường nét đứt bảo toàn một lượng phương sai trung bình (giữa).



Hình 8-7. Lựa chọn không gian con để chiếu

Có vẻ hợp lý khi chọn trục bảo toàn lượng phương sai tối đa, vì nó rất có thể sẽ mất ít thông tin hơn các phép chiếu khác. Một cách khác để biện minh cho lựa chọn này là đó là trục giảm thiểu khoảng cách bình phương trung bình giữa tập dữ liệu gốc và phép chiếu của nó lên trục đó. Đây là ý tưởng khá đơn giản đằng sau PCA.

Các thành phần chính (Principal Components)

PCA xác định trục có phương sai lớn nhất trong tập huấn luyện. Trong Hình 8-7, đó là đường liền nét. Nó cũng tìm một trục thứ hai, trục giao với trục thứ nhất, chiếm phần lớn phương sai còn lại. Trong ví dụ 2D này không có lựa chọn nào khác: đó là đường chấm chấm. Nếu là một tập dữ liệu có chiều cao hơn, PCA cũng sẽ tìm một trục thứ ba, trục giao với cả hai trục trước, và một trục thứ tư, thứ năm, v.v. — nhiều trục bằng số chiều trong tập dữ liệu. Trục thứ i được gọi là **thành phần chính (PC)** thứ i của dữ liệu. Trong Hình 8-7, PC thứ nhất là trục mà vector c_1 nằm trên đó, và PC thứ hai là trục mà vector c_2 nằm trên đó. Trong Hình 8-2, hai PC đầu tiên nằm trên mặt phẳng chiếu, và PC thứ ba là trục trục giao với mặt phẳng đó. Sau phép chiếu, trong Hình 8-3, PC thứ nhất tương ứng với trục z_1 , và PC thứ hai tương ứng với trục z_2 .

Vậy làm thế nào bạn có thể tìm các thành phần chính của một tập huấn luyện? May mắn thay, có một kỹ thuật phân tách ma trận tiêu chuẩn được gọi là **phân tách giá trị số ít (SVD)** có thể phân tách ma trận tập huấn luyện X thành phép nhân ma trận của ba ma trận $U\Sigma V^T$, trong đó V chứa các vector đơn vị định nghĩa tất cả các thành phần chính mà bạn đang tìm kiếm, như trong Phương trình 8-1. Phương trình 8-1. Ma trận các thành phần chính

$$V = \begin{pmatrix} | & | & \cdots & | \\ \mathbf{c}_1 & \mathbf{c}_2 & & \mathbf{c}_n \\ | & | & & | \end{pmatrix}$$

Đoạn mã Python sau sử dụng hàm `svd()` của NumPy để lấy tất cả các thành phần chính của tập huấn luyện 3D được biểu diễn trong Hình 8-2, sau đó nó trích xuất hai vector đơn vị định nghĩa hai PC đầu tiên:

```
import numpy as np

X = [...] # tạo một tập dữ liệu 3D nhỏ
X_centered = X - X.mean(axis=0)
U, s, Vt = np.linalg.svd(X_centered)
c1 = Vt[0]
c2 = Vt[1]
```

Chiếu xuống d chiều

Khi bạn đã xác định tất cả các thành phần chính, bạn có thể giảm chiều của tập dữ liệu xuống d chiều bằng cách chiếu nó lên siêu phẳng được định nghĩa bởi d thành phần chính đầu tiên. Việc chọn siêu phẳng này đảm bảo rằng phép chiếu sẽ bảo toàn càng nhiều phương sai càng tốt. Ví dụ, trong Hình 8-2, tập dữ liệu 3D được chiếu xuống mặt phẳng 2D được định nghĩa bởi hai thành phần chính đầu tiên, bảo toàn một phần lớn phương sai của tập dữ liệu. Kết quả là, phép chiếu 2D trông rất giống tập dữ liệu 3D gốc. Để chiếu tập huấn luyện lên siêu phẳng và thu được tập dữ liệu giảm chiều $X_d - proj$ có chiều d , hãy tính phép nhân ma trận của ma trận tập huấn luyện X với ma trận W_d , được định nghĩa là ma trận chứa d cột đầu tiên của V , như trong Phương trình 8-2. Phương trình 8-2. Chiếu tập huấn luyện xuống d chiều

$$X_{d-proj} = XW_d$$

Đoạn mã Python sau chiếu tập huấn luyện lên mặt phẳng được định nghĩa bởi hai thành phần chính đầu tiên:

```
W2 = Vt[:2].T
X2D = X_centered @ W2
```

Vậy là xong! Bây giờ bạn đã biết cách giảm chiều của bất kỳ tập dữ liệu nào bằng cách chiếu nó xuống bất kỳ số chiều nào, trong khi vẫn bảo toàn càng nhiều phương sai càng tốt.

Sử dụng Scikit-Learn

Lớp PCA của Scikit-Learn sử dụng SVD để triển khai PCA, giống như chúng ta đã làm ở đầu chương này. Đoạn mã sau đây áp dụng PCA để giảm chiều của tập dữ liệu xuống hai chiều (lưu ý rằng nó tự động xử lý việc căn giữa dữ liệu):

```
from sklearn.decomposition import PCA

pca = PCA(n_components=2)
X2D = pca.fit_transform(X)
```

Sau khi khớp bộ biến đổi PCA với tập dữ liệu, thuộc tính `components_` của nó chứa ma trận chuyển vị của W_d : nó chứa một hàng cho mỗi trong số d thành phần chính đầu tiên.

Tỷ lệ phương sai giải thích (Explained Variance Ratio)

Một thông tin hữu ích khác là **tỷ lệ phương sai giải thích** của mỗi thành phần chính, có sẵn thông qua biến `explained_variance_ratio_`. Tỷ lệ này chỉ ra tỷ lệ phương sai của tập dữ liệu nằm dọc theo mỗi thành phần chính. Ví dụ, hãy xem tỷ lệ phương sai giải thích của hai thành phần đầu tiên của tập dữ liệu 3D được biểu diễn trong Hình 8-2:

```
>>> pca.explained_variance_ratio_  
array([0.7578477 , 0.15186921])
```

Kết quả này cho chúng ta biết rằng khoảng 76% phương sai của tập dữ liệu nằm dọc theo PC thứ nhất, và khoảng 15% nằm dọc theo PC thứ hai. Điều này để lại khoảng 9% cho PC thứ ba, vì vậy có lý khi giả định rằng PC thứ ba có lẽ mang rất ít thông tin.

Chọn số chiều phù hợp

Thay vì tùy ý chọn số chiều để giảm xuống, đơn giản hơn là chọn số chiều mà tổng hợp lại chiếm một phần đủ lớn của phương sai—ví dụ, 95% (Một ngoại lệ của quy tắc này, tất nhiên, là nếu bạn đang giảm chiều để trực quan hóa dữ liệu, trong trường hợp đó bạn sẽ muốn giảm chiều xuống 2 hoặc 3). Đoạn mã sau đây tải và chia tập dữ liệu MNIST (được giới thiệu trong Chương 3) và thực hiện PCA mà không giảm chiều, sau đó tính toán số chiều tối thiểu cần thiết để bảo toàn 95% phương sai của tập huấn luyện:

```
from sklearn.datasets import fetch_openml  
  
mnist = fetch_openml('mnist_784', as_frame=False)  
X_train, y_train = mnist.data[:60_000], mnist.target[:60_000]  
X_test, y_test = mnist.data[60_000:], mnist.target[60_000:]  
  
pca = PCA()  
pca.fit(X_train)  
  
cumsum = np.cumsum(pca.explained_variance_ratio_)  
d = np.argmax(cumsum >= 0.95) + 1 # d bằng 154
```

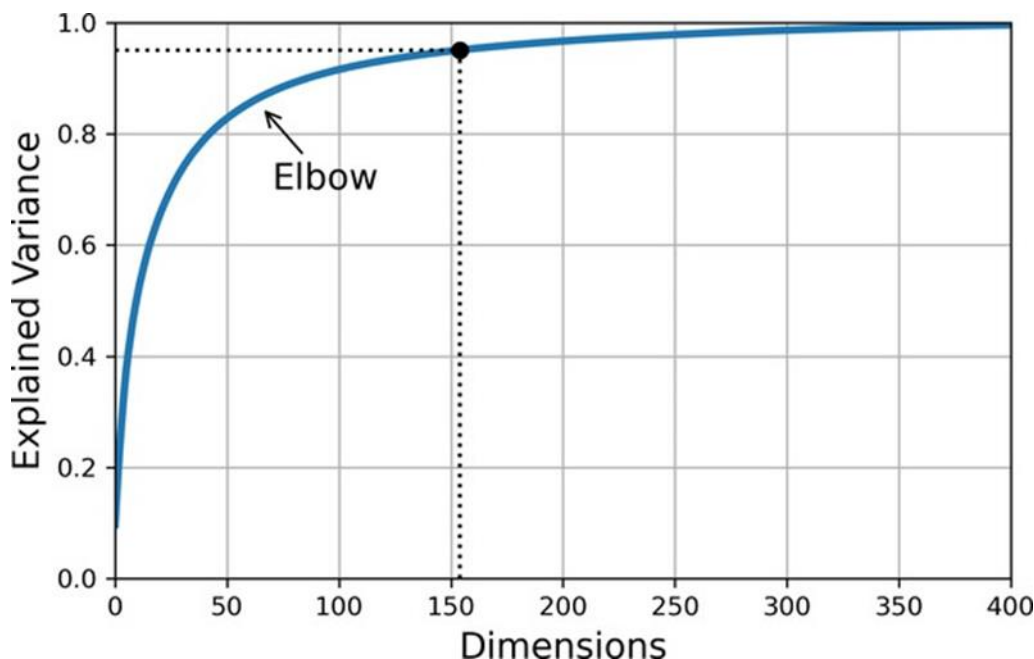
Sau đó, bạn có thể đặt `n_components=d` và chạy lại PCA, nhưng có một lựa chọn tốt hơn. Thay vì chỉ định số lượng thành phần chính bạn muốn bảo toàn, bạn có thể đặt `n_components` là một số thực từ 0.0 đến 1.0, cho biết tỷ lệ phương sai bạn muốn bảo toàn:

```
pca = PCA(n_components=0.95)  
X_reduced = pca.fit_transform(X_train)
```

Số lượng thành phần thực tế được xác định trong quá trình huấn luyện, và nó được lưu trữ trong thuộc tính `n_components_`:

```
>>> pca.n_components_  
154
```

Một lựa chọn khác là vẽ biểu đồ phương sai giải thích như một hàm của số chiều (chỉ cần vẽ cumsum; xem Hình 8-8). Thường sẽ có một điểm uốn trên đường cong, nơi phương sai giải thích dừng tăng nhanh. Trong trường hợp này, bạn có thể thấy rằng việc giảm chiều xuống khoảng 100 chiều sẽ không mất quá nhiều phương sai giải thích.



Hình 8-8. Phương sai giải thích như một hàm của số chiều

Cuối cùng, nếu bạn đang sử dụng giảm chiều như một bước tiền xử lý cho một tác vụ học có giám sát (ví dụ: phân loại), thì bạn có thể điều chỉnh số lượng chiều như bất kỳ siêu tham số nào khác (xem Chương 2). Ví dụ, đoạn mã sau đây tạo một pipeline hai bước, đầu tiên giảm chiều bằng PCA, sau đó phân loại bằng rừng ngẫu nhiên. Tiếp theo, nó sử dụng RandomizedSearchCV để tìm một sự kết hợp tốt các siêu tham số cho cả PCA và bộ phân loại rừng ngẫu nhiên. Ví dụ này thực hiện một tìm kiếm nhanh, chỉ điều chỉnh 2 siêu tham số, huấn luyện trên chỉ 1.000 trường hợp và chạy trong chỉ 10 lần lặp, nhưng bạn có thể thoải mái thực hiện một tìm kiếm kỹ lưỡng hơn nếu có thời gian:

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import RandomizedSearchCV
from sklearn.pipeline import make_pipeline

clf = make_pipeline(PCA(random_state=42),
                    RandomForestClassifier(random_state=42))
param_distrib = {
    "pca__n_components": np.arange(10, 80),
    "randomforestclassifier__n_estimators": np.arange(50, 500)
}
rnd_search = RandomizedSearchCV(clf, param_distrib, n_iter=10, cv=3,
                                random_state=42)
rnd_search.fit(X_train[:1000], y_train[:1000])
```

Hãy xem các siêu tham số tốt nhất được tìm thấy:

```
>>> print(rnd_search.best_params_)  
{'randomforestclassifier__n_estimators': 465, 'pca__n_components': 23}
```

Thật thú vị khi lưu ý số lượng thành phần tối ưu thấp như thế nào: chúng ta đã giảm một tập dữ liệu 784 chiều xuống chỉ còn 23 chiều! Điều này liên quan đến việc chúng ta đã sử dụng rừng ngẫu nhiên, một mô hình khá mạnh mẽ. Nếu chúng ta sử dụng một mô hình tuyến tính thay thế, chẳng hạn như `SGDClassifier`, tìm kiếm sẽ cho thấy chúng ta cần bảo toàn nhiều chiều hơn (khoảng 70).

PCA để nén

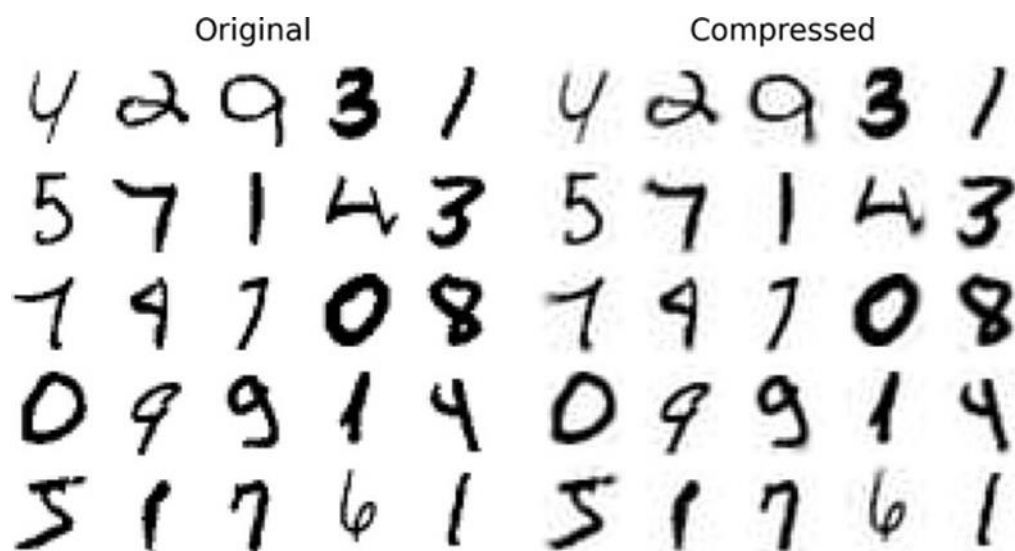
Sau khi giảm chiều dữ liệu, tập huấn luyện chiếm ít không gian hơn nhiều. Ví dụ, sau khi áp dụng PCA cho tập dữ liệu MNIST trong khi vẫn bảo toàn 95% phương sai của nó, chúng ta còn lại 154 đặc trưng, thay vì 784 đặc trưng ban đầu. Do đó, tập dữ liệu giờ đây nhỏ hơn 20% kích thước ban đầu của nó, và chúng ta chỉ mất 5% phương sai của nó! Đây là một tỷ lệ nén hợp lý, và dễ dàng nhận thấy việc giảm kích thước như vậy sẽ tăng tốc thuật toán phân loại đáng kể.

Cũng có thể giải nén tập dữ liệu đã giảm chiều trở lại 784 chiều bằng cách áp dụng phép biến đổi ngược của phép chiếu PCA. Điều này sẽ không trả lại dữ liệu gốc cho bạn, vì phép chiếu đã mất một chút thông tin (trong phạm vi 5% phương sai đã bị loại bỏ), nhưng nó có thể sẽ gần với dữ liệu gốc. Khoảng cách bình phương trung bình giữa dữ liệu gốc và dữ liệu được tái tạo (nén và sau đó giải nén) được gọi là **sai số tái tạo**.

Phương thức `inverse_transform()` cho phép chúng ta giải nén tập dữ liệu MNIST đã giảm chiều trở lại 784 chiều:

```
X_recovered = pca.inverse_transform(X_reduced)
```

Hình 8-9 cho thấy một vài chữ số từ tập huấn luyện gốc (bên trái), và các chữ số tương ứng sau khi nén và giải nén. Bạn có thể thấy rằng có một chút mất mát chất lượng hình ảnh, nhưng các chữ số vẫn hầu hết nguyên vẹn.



Hình 8-9. Nén MNIST bảo toàn 95% phương sai

Phương trình cho phép biến đổi ngược được hiển thị trong Phương trình 8-3.

Phương trình 8-3. Phép biến đổi ngược PCA, trở lại số chiều gốc

$$X_{recovered} = X_{d-proj} W_d^T$$

PCA ngẫu nhiên (Randomized PCA)

Nếu bạn đặt siêu tham số `svd_solver` thành “randomized”, Scikit-Learn sử dụng một thuật toán ngẫu nhiên được gọi là **randomized PCA** nhanh chóng tìm thấy một xấp xỉ của d thành phần chính đầu tiên. Độ phức tạp tính toán của nó là $O(m \times d^2) + O(d^3)$, thay vì $O(m \times n^2) + O(n^3)$ cho phương pháp SVD đầy đủ, vì vậy nó nhanh hơn đáng kể so với SVD đầy đủ khi d nhỏ hơn nhiều so với n :

```
rnd_pca = PCA(n_components=154, svd_solver="randomized", random_state=42)
X_reduced = rnd_pca.fit_transform(X_train)
```

PCA tăng dần (Incremental PCA)

Một vấn đề với các triển khai PCA trước đó là chúng yêu cầu toàn bộ tập huấn luyện phải nằm trong bộ nhớ để thuật toán chạy. May mắn thay, các thuật toán **incremental PCA (IPCA)** đã được phát triển cho phép bạn chia tập huấn luyện thành các mini-batch và cung cấp chúng từng mini-batch một. Điều này hữu ích cho các tập huấn luyện lớn và để áp dụng PCA trực tuyến (tức là ngay lập tức, khi các trường hợp mới đến).

Đoạn mã sau chia tập huấn luyện MNIST thành 100 mini-batch (sử dụng hàm `array_split()` của NumPy) và cung cấp chúng cho lớp `IncrementalPCA` của Scikit-Learn để giảm chiều của tập dữ liệu MNIST xuống 154 chiều, giống như trước đây. Lưu ý rằng bạn phải gọi phương thức `partial_fit()` với mỗi mini-batch, thay vì phương thức `fit()` với toàn bộ tập huấn luyện:

```

from sklearn.decomposition import IncrementalPCA

n_batches = 100
inc_pca = IncrementalPCA(n_components=154)

for X_batch in np.array_split(X_train, n_batches):
    inc_pca.partial_fit(X_batch)

X_reduced = inc_pca.transform(X_train)

```

Thay vào đó, bạn có thể sử dụng lớp memmap của NumPy, cho phép bạn thao tác một mảng lớn được lưu trữ trong một tệp nhị phân trên đĩa như thể nó hoàn toàn nằm trong bộ nhớ; lớp này chỉ tải dữ liệu nó cần vào bộ nhớ, khi nó cần. Để chứng minh điều này, trước tiên hãy tạo một tệp ánh xạ bộ nhớ (memmap) và sao chép tập huấn luyện MNIST vào đó, sau đó gọi `flush()` để đảm bảo rằng bất kỳ dữ liệu nào còn trong bộ đệm đều được lưu vào đĩa. Trong thực tế, `X_train` thường sẽ không vừa trong bộ nhớ, vì vậy bạn sẽ tải nó từng đoạn và lưu mỗi đoạn vào đúng phần của mảng memmap:

```

filename = "my_mnist.mmap"
X_mmap = np.memmap(filename, dtype='float32', mode='write', shape=X_train.shape)
X_mmap[:] = X_train #có thể là một vòng lặp thay vào đó, lưu dữ liệu từng đoạn
X_mmap.flush()

```

Tiếp theo, chúng ta có thể tải tệp memmap và sử dụng nó như một mảng NumPy thông thường. Hãy sử dụng lớp `IncrementalPCA` để giảm chiều của nó. Vì thuật toán này chỉ sử dụng một phần nhỏ của mảng tại bất kỳ thời điểm nào, việc sử dụng bộ nhớ vẫn được kiểm soát. Điều này giúp có thể gọi phương thức `fit()` thông thường thay vì `partial_fit()`, điều này khá tiện lợi:

```

X_mmap = np.memmap(filename, dtype="float32", mode="readonly").reshape(-1, 784)
batch_size = X_mmap.shape[0] // n_batches
inc_pca = IncrementalPCA(n_components=154, batch_size=batch_size)
inc_pca.fit(X_mmap)

```

Đối với các tập dữ liệu có chiều rất cao, PCA có thể quá chậm. Như bạn đã thấy trước đó, ngay cả khi bạn sử dụng randomized PCA, độ phức tạp tính toán của nó vẫn là $O(m \times d^2) + O(d^3)$, vì vậy số chiều mục tiêu d không được quá lớn. Nếu bạn đang xử lý một tập dữ liệu với hàng chục nghìn đặc trưng trở lên (ví dụ: hình ảnh), thì quá trình huấn luyện có thể trở nên quá chậm: trong trường hợp này, bạn nên xem xét sử dụng phép chiếu ngẫu nhiên thay thế.

Phép chiếu ngẫu nhiên (Random Projection)

Như tên gọi của nó, thuật toán **phép chiếu ngẫu nhiên** chiếu dữ liệu vào một không gian có chiều thấp hơn bằng cách sử dụng phép chiếu tuyến tính ngẫu nhiên. Điều này nghe có vẻ

điên rồ, nhưng hóa ra một phép chiếu ngẫu nhiên như vậy thực sự rất có khả năng bảo toàn khoảng cách khá tốt, như đã được chứng minh toán học bởi William B. Johnson và Joram Lindenstrauss trong một định lý nổi tiếng. Do đó, hai trường hợp tương tự sẽ vẫn tương tự sau phép chiếu, và hai trường hợp rất khác nhau sẽ vẫn rất khác nhau.

Rõ ràng, bạn càng giảm nhiều chiều, càng mất nhiều thông tin và khoảng cách càng bị biến dạng. Vậy làm thế nào bạn có thể chọn số chiều tối ưu? Vâng, Johnson và Lindenstrauss đã đưa ra một phương trình xác định số chiều tối thiểu cần bảo toàn để đảm bảo—với xác suất cao—rằng khoảng cách sẽ không thay đổi quá một dung sai nhất định. Ví dụ, nếu bạn có một tập dữ liệu chứa $m = 5.000$ trường hợp với $n = 20.000$ đặc trưng mỗi trường hợp, và bạn không muốn khoảng cách bình phương giữa bất kỳ hai trường hợp nào thay đổi quá $\epsilon = 10$, thì bạn nên chiếu dữ liệu xuống d chiều, với $d \geq 4 \log(m) / (\frac{1}{2}\epsilon^2 - \frac{1}{3}\epsilon^3)$, là 7.300 chiều. Đó là một sự giảm chiều khá đáng kể!

Lưu ý rằng phương trình không sử dụng n , nó chỉ dựa vào m và ϵ . Phương trình này được triển khai bởi hàm `johnson_lindenstrauss_min_dim()`:

```
>>> from sklearn.random_projection import johnson_lindenstrauss_min_dim
>>> m, ε = 5_000, 0.1
>>> d = johnson_lindenstrauss_min_dim(m, eps=ε)
>>> d
7300
```

Bây giờ chúng ta chỉ có thể tạo một ma trận ngẫu nhiên P có hình dạng $[d, n]$, trong đó mỗi phần tử được lấy mẫu ngẫu nhiên từ phân phối Gaussian với giá trị trung bình 0 và phương sai $1/d$, và sử dụng nó để chiếu một tập dữ liệu từ n chiều xuống d :

```
n = 20_000
np.random.seed(42)
P = np.random.randn(d, n) / np.sqrt(d) # độ lệch chuẩn = căn bậc hai của phương sai

X = np.random.randn(m, n) # tạo một tập dữ liệu giả
X_reduced = X @ P.T
```

Chỉ có thể thôi! Nó đơn giản và hiệu quả, và không yêu cầu huấn luyện: điều duy nhất thuật toán cần để tạo ma trận ngẫu nhiên là hình dạng của tập dữ liệu. Bản thân dữ liệu không được sử dụng chút nào.

Scikit-Learn cung cấp lớp `GaussianRandomProjection` để làm chính xác những gì chúng ta vừa làm: khi bạn gọi phương thức `fit()` của nó, nó sử dụng `johnson_lindenstrauss_min_dim()` để xác định chiều đầu ra, sau đó nó tạo một ma trận ngẫu nhiên, mà nó lưu trữ trong thuộc tính `components_`. Sau đó, khi bạn gọi `transform()`, nó sử dụng ma trận này để thực hiện phép chiếu. Khi tạo bộ biến đổi, bạn có thể đặt `eps` nếu bạn muốn điều chỉnh *epsilon* (mặc định là 0.1), và `n_components` nếu bạn muốn buộc một chiều mục tiêu d cụ thể. Đoạn mã sau đây cho kết quả tương tự như đoạn mã trước đó (bạn cũng có thể xác minh rằng `gaussian_rnd_proj.components_` bằng P):

```
from sklearn.random_projection import GaussianRandomProjection

gaussian_rnd_proj = GaussianRandomProjection(eps=ε, random_state=42)
X_reduced = gaussian_rnd_proj.fit_transform(X) # cùng kết quả như trên
```

Scikit-Learn cũng cung cấp một bộ biến đổi phép chiếu ngẫu nhiên thứ hai, được gọi là `SparseRandomProjection`. Nó xác định chiều mục tiêu theo cùng một cách, tạo một ma trận ngẫu nhiên có cùng hình dạng và thực hiện phép chiếu giống hệt nhau. Sự khác biệt chính là ma trận ngẫu nhiên thưa thớt. Điều này có nghĩa là nó sử dụng ít bộ nhớ hơn nhiều: khoảng 25 MB thay vì gần 1.2 GB trong ví dụ trước đó! Và nó cũng nhanh hơn nhiều, cả để tạo ma trận ngẫu nhiên và để giảm chiều: nhanh hơn khoảng 50% trong trường hợp này. Hơn nữa, nếu đầu vào thưa thớt, phép biến đổi vẫn giữ nó thưa thớt (trừ khi bạn đặt `dense_output=True`). Cuối cùng, nó có cùng thuộc tính bảo toàn khoảng cách như cách tiếp cận trước, và chất lượng giảm chiều có thể so sánh được. Tóm lại, thường nên sử dụng bộ biến đổi này thay vì bộ biến đổi đầu tiên, đặc biệt đối với các tập dữ liệu lớn hoặc thưa thớt.

Tỷ lệ r các phần tử khác không trong ma trận ngẫu nhiên thưa thớt được gọi là **mật độ** của nó. Theo mặc định, nó bằng $1/\sqrt{rtn}$. Với 20.000 đặc trưng, điều này có nghĩa là chỉ 1 trong khoảng 141 ô trong ma trận ngẫu nhiên là khác không: điều đó khá thưa thớt! Bạn có thể đặt siêu tham số `density` thành một giá trị khác nếu bạn muốn. Mỗi ô trong ma trận ngẫu nhiên thưa thớt có xác suất r là khác không, và mỗi giá trị khác không là $-v$ hoặc $+v$ (cả hai đều có khả năng như nhau), trong đó $v = 1/\sqrt{rtdr}$.

Nếu bạn muốn thực hiện phép biến đổi ngược, trước tiên bạn cần tính nghịch đảo giả của ma trận thành phần bằng hàm `pinv()` của SciPy, sau đó nhân dữ liệu đã giảm chiều với ma trận chuyển vị của nghịch đảo giả:

```
components_pinv = np.linalg.pinv(gaussian_rnd_proj.components_)
X_recovered = X_reduced @ components_pinv.T
```

Tóm lại, phép chiếu ngẫu nhiên là một thuật toán giảm chiều đơn giản, nhanh, hiệu quả bộ nhớ và mạnh mẽ đáng ngạc nhiên mà bạn nên ghi nhớ, đặc biệt khi bạn xử lý các tập dữ liệu có chiều cao.

Nhúng tuyến tính cục bộ

Nhúng tuyến tính cục bộ (LLE) là một kỹ thuật giảm chiều phi tuyến tính (NLDR). Đây là một kỹ thuật học đa tạp không dựa vào phép chiếu, không giống như PCA và phép chiếu ngẫu nhiên. Tóm lại, LLE hoạt động bằng cách đầu tiên đo lường cách mỗi trường hợp huấn luyện liên quan tuyến tính đến các láng giềng gần nhất của nó, sau đó tìm kiếm một biểu diễn chiều thấp của tập huấn luyện nơi các mối quan hệ cục bộ này được bảo toàn tốt nhất (chi tiết hơn sẽ được trình bày ngay sau đây). Cách tiếp cận này làm cho nó đặc biệt tốt trong việc mở cuộn các đa tạp bị xoắn, đặc biệt khi không có quá nhiều nhiễu.

Đoạn mã sau tạo một Swiss roll, sau đó sử dụng lớp `LocallyLinearEmbedding` của Scikit-Learn để mở cuộn nó:

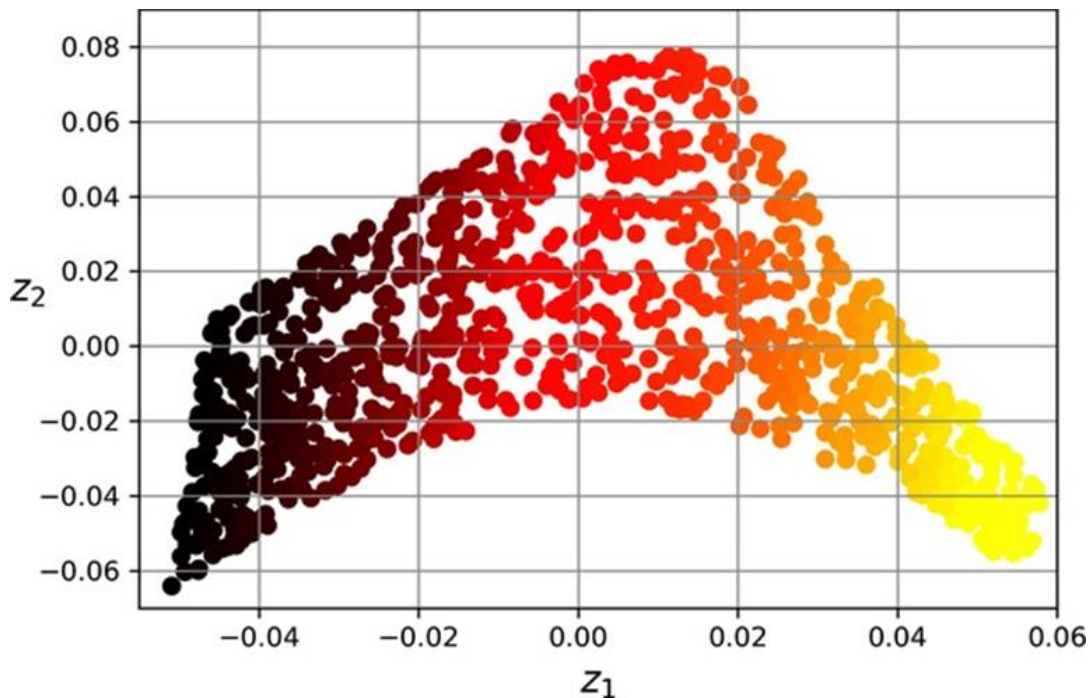
```

from sklearn.datasets import make_swiss_roll
from sklearn.manifold import LocallyLinearEmbedding

X_swiss, t = make_swiss_roll(n_samples=1000, noise=0.2, random_state=42)
lle = LocallyLinearEmbedding(n_components=2, n_neighbors=10, random_state=42)
X_unrolled = lle.fit_transform(X_swiss)

```

Biến t là một mảng NumPy 1D chứa vị trí của mỗi trường hợp dọc theo trục cuộn của Swiss roll. Chúng ta không sử dụng nó trong ví dụ này, nhưng nó có thể được sử dụng làm mục tiêu cho một tác vụ hồi quy phi tuyến. Tập dữ liệu 2D thu được được hiển thị trong Hình 8-10. Như bạn có thể thấy, Swiss roll được mở cuộn hoàn toàn và khoảng cách giữa các trường hợp được bảo toàn tốt cục bộ. Tuy nhiên, khoảng cách không được bảo toàn ở quy mô lớn hơn: Swiss roll đã mở cuộn phải là một hình chữ nhật, không phải loại dải bị kéo dài và xoắn này. Tuy nhiên, LLE đã làm khá tốt việc mô hình hóa đa tạp.



Hình 8-10. Swiss roll đã mở cuộn bằng LLE

Bước 1: Mô hình hóa tuyến tính các mối quan hệ cục bộ

Đối với mỗi trường hợp huấn luyện $x^{(i)}$, thuật toán xác định k láng giềng gần nhất của nó. Sau đó, nó cố gắng tái tạo $x^{(i)}$ dưới dạng một hàm tuyến tính của các láng giềng này. Cụ thể hơn, thuật toán tìm các trọng số $w_{i,j}$ sao cho khoảng cách bình phương giữa $x^{(i)}$ và $\sum_{j=1}^m w_{i,j} x^{(j)}$ là nhỏ nhất có thể. Giả sử rằng $w_{i,j} = 0$ nếu $x^{(j)}$ không phải là một trong k láng giềng gần nhất của $x^{(i)}$.

Công thức 8-4: Bước 1 của LLE: Mô hình hóa tuyến tính các mối quan hệ cục bộ

$$\begin{aligned} \widehat{\mathbf{W}} = \underset{\mathbf{W}}{\operatorname{argmin}} \sum_{i=1}^m \left(\mathbf{x}^{(i)} - \sum_{j=1}^m w_{i,j} \mathbf{x}^{(j)} \right)^2 \\ \text{subject to } \begin{cases} w_{i,j} = 0 & \text{if } \mathbf{x}^{(j)} \text{ is not one of the } k \text{ n.n. of } \mathbf{x}^{(i)} \\ \sum_{j=1}^m w_{i,j} = 1 & \text{for } i = 1, 2, \dots, m \end{cases} \end{aligned}$$

Sau bước này, ma trận trọng số $\widehat{\mathbf{W}}$ mã hóa các mối quan hệ tuyến tính cục bộ giữa các trường hợp huấn luyện.

Bước 2: Giảm chiều dữ liệu trong khi vẫn giữ các mối quan hệ

Bước thứ hai là ánh xạ các trường hợp huấn luyện vào một không gian d chiều (với $d < n$) trong khi vẫn bảo toàn các mối quan hệ cục bộ này càng nhiều càng tốt. Nếu $\mathbf{z}^{(i)}$ là hình ảnh của $\mathbf{x}^{(i)}$ trong không gian d chiều, thì chúng ta muốn khoảng cách bình phương giữa $\mathbf{z}^{(i)}$ và $\sum_{j=1}^m w_{i,j} \mathbf{z}^{(j)}$ nhỏ nhất có thể. Ý tưởng này dẫn đến bài toán tối ưu hóa không ràng buộc được mô tả trong Công thức 8-5.

Công thức 8-5: Bước 2 của LLE: Giảm chiều dữ liệu trong khi vẫn giữ các mối quan hệ

$$\widehat{\mathbf{Z}} = \underset{\mathbf{Z}}{\operatorname{argmin}} \sum_{i=1}^m \left(\mathbf{z}^{(i)} - \sum_{j=1}^m \widehat{w}_{i,j} \mathbf{z}^{(j)} \right)^2$$

Lưu ý rằng $\mathbf{Z}$ là ma trận chứa tất cả các $\mathbf{z}^{(i)}$.

Việc triển khai LLE của Scikit-Learn có độ phức tạp tính toán như sau: $O(m \log(m) n \log(k))$ để tìm các láng giềng gần nhất, $O(mnk^3)$ để tối ưu hóa trọng số, và $O(dm^2)$ để xây dựng các biểu diễn chiều thấp. Thật không may, số hạng m^2 trong thuật ngữ cuối cùng làm cho thuật toán này mở rộng kém đối với các tập dữ liệu rất lớn.

Các kỹ thuật giảm chiều khác

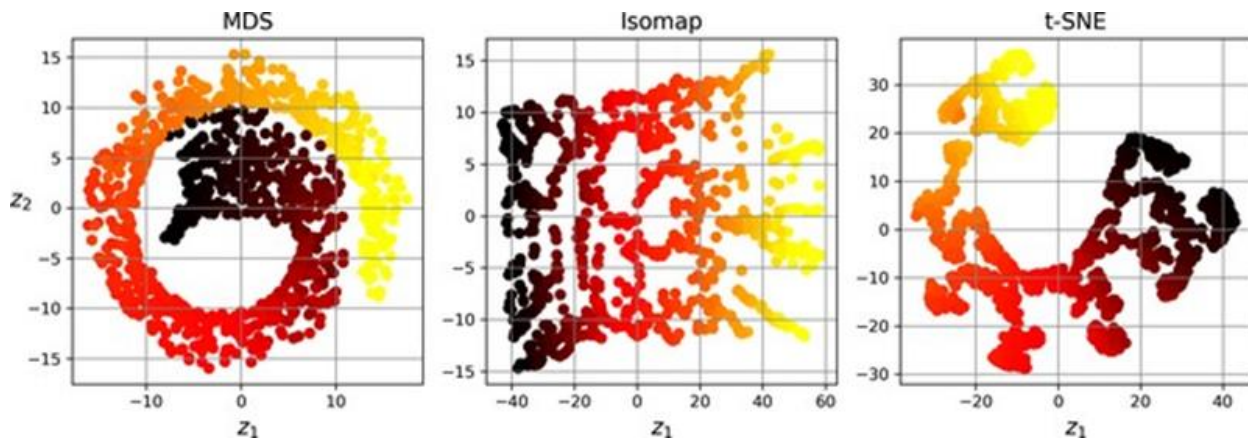
Trước khi chúng ta kết thúc chương này, hãy xem nhanh một vài kỹ thuật giảm chiều phổ biến khác có sẵn trong Scikit-Learn:

- `sklearn.manifold.MDS` **Multidimensional scaling (MDS)** giảm chiều dữ liệu trong khi cố gắng bảo toàn khoảng cách giữa các trường hợp. Phép chiếu ngẫu nhiên làm điều đó cho dữ liệu có chiều cao, nhưng nó không hoạt động tốt trên dữ liệu có chiều thấp.
- `sklearn.manifold.Isomap` **Isomap** tạo một đồ thị bằng cách nối mỗi trường hợp với các láng giềng gần nhất của nó, sau đó giảm chiều trong khi cố gắng bảo toàn khoảng

cách trắc địa giữa các trường hợp. Khoảng cách trắc địa giữa hai nút trong một đồ thị là số nút trên đường đi ngắn nhất giữa các nút này.

- `sklearn.manifold.TSNE` **t-distributed stochastic neighbor embedding (t-SNE)** giảm chiều trong khi cố gắng giữ các trường hợp tương tự gần nhau và các trường hợp không tương tự xa nhau. Nó chủ yếu được sử dụng để trực quan hóa, đặc biệt để trực quan hóa các cụm trường hợp trong không gian chiều cao. Ví dụ, trong các bài tập ở cuối chương này, bạn sẽ sử dụng t-SNE để trực quan hóa một bản đồ 2D của các hình ảnh MNIST.
- `sklearn.discriminant_analysis.LinearDiscriminantAnalysis` **Linear discriminant analysis (LDA)** là một thuật toán phân loại tuyến tính mà, trong quá trình huấn luyện, học các trục phân biệt nhất giữa các lớp. Các trục này sau đó có thể được sử dụng để định nghĩa một siêu phẳng để chiếu dữ liệu lên đó. Lợi ích của cách tiếp cận này là phép chiếu sẽ giữ các lớp cách xa nhau nhất có thể, vì vậy LDA là một kỹ thuật tốt để giảm chiều trước khi chạy một thuật toán phân loại khác (trừ khi LDA một mình là đủ).

Hình 8-11 cho thấy kết quả của MDS, Isomap và t-SNE trên Swiss roll. MDS quản lý để làm phẳng Swiss roll mà không làm mất độ cong tổng thể của nó, trong khi Isomap loại bỏ hoàn toàn nó. Tùy thuộc vào tác vụ tiếp theo, việc bảo toàn cấu trúc quy mô lớn có thể tốt hoặc xấu. t-SNE thực hiện một công việc khá tốt trong việc làm phẳng Swiss roll, bảo toàn một chút độ cong, và nó cũng khuếch đại các cụm, xé cuộn ra. Một lần nữa, điều này có thể tốt hoặc xấu, tùy thuộc vào tác vụ tiếp theo.



Hình 8-11. Sử dụng các kỹ thuật khác nhau để giảm Swiss roll xuống 2D

Bài tập

1. Động lực chính để giảm chiều dữ liệu là gì? Những hạn chế chính là gì?
2. Lời nguyên của số chiều là gì?
3. Một khi chiều của tập dữ liệu đã được giảm, có thể đảo ngược thao tác không? Nếu có, bằng cách nào? Nếu không, tại sao?
4. PCA có thể được sử dụng để giảm chiều của một tập dữ liệu phi tuyến tính cao không?

5. Giả sử bạn thực hiện PCA trên một tập dữ liệu 1.000 chiều, đặt tỷ lệ phương sai giải thích là 95%. Tập dữ liệu kết quả sẽ có bao nhiêu chiều?
6. Trong những trường hợp nào bạn sẽ sử dụng PCA thông thường, PCA tăng dần, PCA ngẫu nhiên, hoặc phép chiếu ngẫu nhiên?
7. Làm thế nào bạn có thể đánh giá hiệu suất của một thuật toán giảm chiều trên tập dữ liệu của bạn?
8. Có ý nghĩa gì khi nối chuỗi hai thuật toán giảm chiều khác nhau không?
9. Tải tập dữ liệu MNIST (được giới thiệu trong Chương 3) và chia nó thành một tập huấn luyện và một tập kiểm tra (lấy 60.000 trường hợp đầu tiên để huấn luyện, và 10.000 còn lại để kiểm tra). Huấn luyện một bộ phân loại rừng ngẫu nhiên trên tập dữ liệu và tính thời gian mất bao lâu, sau đó đánh giá mô hình kết quả trên tập kiểm tra. Tiếp theo, sử dụng PCA để giảm chiều của tập dữ liệu, với tỷ lệ phương sai giải thích là 95%. Huấn luyện một bộ phân loại rừng ngẫu nhiên mới trên tập dữ liệu đã giảm chiều và xem mất bao lâu. Quá trình huấn luyện có nhanh hơn nhiều không? Tiếp theo, đánh giá bộ phân loại trên tập kiểm tra. Nó so sánh như thế nào với bộ phân loại trước đó? Thử lại với một `SGDClassifier`. PCA giúp được bao nhiêu bây giờ?
10. Sử dụng t-SNE để giảm 5.000 hình ảnh đầu tiên của tập dữ liệu MNIST xuống 2 chiều và vẽ kết quả bằng Matplotlib. Bạn có thể sử dụng một biểu đồ phân tán (scatterplot) với 10 màu khác nhau để biểu thị lớp mục tiêu của mỗi hình ảnh. Thay vào đó, bạn có thể thay thế mỗi chấm trong biểu đồ phân tán bằng lớp của trường hợp tương ứng (một chữ số từ 0 đến 9), hoặc thậm chí vẽ các phiên bản thu nhỏ của chính các hình ảnh chữ số (nếu bạn vẽ tất cả các chữ số, trực quan hóa sẽ quá lộn xộn, vì vậy bạn nên vẽ một mẫu ngẫu nhiên hoặc chỉ vẽ một trường hợp nếu không có trường hợp nào khác đã được vẽ ở khoảng cách gần). Bạn sẽ nhận được một hình ảnh trực quan đẹp với các cụm chữ số được phân tách tốt. Thử sử dụng các thuật toán giảm chiều khác, chẳng hạn như PCA, LLE hoặc MDS, và so sánh các hình ảnh trực quan thu được. Các giải pháp cho các bài tập này có sẵn ở cuối sổ tay của chương này, tại <https://homl.info/colab3>.